

Totem Inteligente Inclusivo

Challenge Flexmedia

Matheus Meissner - RM567080

Turma - FIAP • Artificial Intelligence & Machine Learning

Entrega – Integração Sensores + SQL + Análise + Dashboard

Github - https://github.com/matheus-meissner/inclusive_culture_totem.git

YouTube - <https://youtu.be/KuR6zuKEfyU>

Sumário

- 1. Visão Geral da Sprint 2.....4
- 2. Arquitetura Implementada4
- 3. Fluxo de Dados (Entrada → Processamento → Saída)5
 - 3.1. Entrada (Simulador dos Sensores)5
 - 3.2. Processamento (Banco SQL + Python)5
 - 3.3. Saída (Insights, Predições e Dashboard)5
- 4. Módulos Técnicos (Descrição Completa).....6
 - 4.1 Módulo 1 — Simulador de Sensores.....6
 - 4.2 Módulo 2 — Armazenamento SQL (SQLite).....7
 - 4.3 Módulo 3 — Análise de Dados (Notebook).....7
 - 4.4 Módulo 4 — Machine Learning Supervisionado 12
 - 4.5 Módulo 5 — Dashboard em Streamlit..... 13
- 5. Conclusão Técnica da Sprint 2 15

Controle de Versão

Data	Versão	Elaborado por	Descrição
23/11/2025	1.0	Matheus Meissner	Elaboração do documento

1. Visão Geral da Sprint 2

Nesta sprint, desenvolvi a primeira integração funcional do Totem Inteligente Inclusivo, conectando a simulação dos sensores a um fluxo completo de armazenamento, análise estatística, Machine Learning e visualização.

O objetivo foi validar, de ponta a ponta, como o totem coleta dados, como esses dados são estruturados, e de que forma eles se transformam em métricas e insights úteis.

A Sprint 2 representa a etapa em que o sistema começa a “ganhar vida”: registros reais (ou simulados) passam a ser persistidos, analisados e exibidos em tempo quase real exatamente como um totem físico faria em um ambiente cultural ou bibliotecário.

Observação: O modelo supervisionado é demonstrativo e não representa um sistema real de classificação do totem. O objetivo aqui é somente validar o pipeline de dados da Sprint 2.

2. Arquitetura Implementada

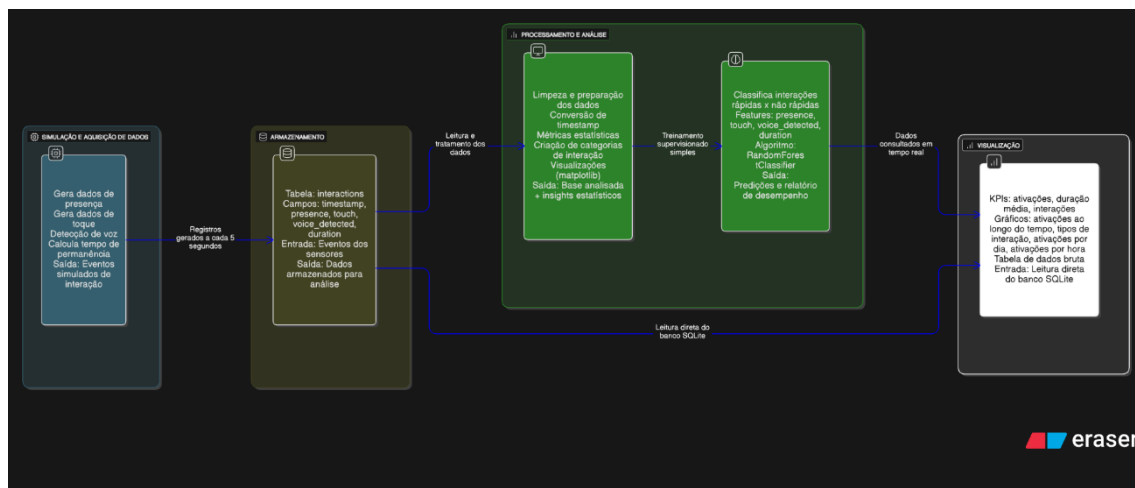
A arquitetura desta sprint é inteiramente local, respeitando o escopo solicitado pela Flexmedia: integrar sensores (simulados), banco SQL simples e análise Python.

O fluxo da Sprint 2 é composto por cinco módulos principais:

1. **Simulação e Aquisição de Dados** — geração contínua de eventos simulados (presença, toque, voz, duração).
2. **Armazenamento** — banco de dados SQLite (totem.db).
3. **Processamento e Análise** — notebook Python para limpeza, métricas e visualizações.
4. **Machine Learning Supervisionado** — classificador simples de tipo de interação.
5. **Dashboard Interativo** — painel Streamlit consultando o banco em tempo real.

O diagrama abaixo representa o fluxo completo (o mesmo criado no Eraser, que vai no PDF):

Simulação → Banco SQL → Notebook → ML → Dashboard



3. Fluxo de Dados (Entrada → Processamento → Saída)

3.1. Entrada (Simulador dos Sensores)

Os dados simulados representam eventos reais de utilização do totem. A cada 5 segundos, o sistema gera um pacote contendo:

- presence → 1 quando alguém está diante do totem
- touch → 1 quando há toque detectado
- voice_detected → 1 quando há comando de voz
- duration → tempo de permanência em segundos
- timestamp → registro ISO do momento da interação

Esse fluxo representa de forma fiel o comportamento esperado do ESP32 + sensores reais.

3.2. Processamento (Banco SQL + Python)

Assim que o simulador gera dados, eles são enviados para o banco SQLite.

O notebook (analysis.ipynb) executa:

- leitura e ordenação das interações
- conversão de timestamp
- remoção de duplicados
- tratamento de inconsistências (ex.: duração negativa)
- cálculos estatísticos
- criação de categorias de interação (rápida, média, longa, sem interação)

3.3. Saída (Insights, Predições e Dashboard)

Após a análise:

1. gráficos são gerados (ativação por dia, hora, evolução temporal)
2. métricas de uso são calculadas (tempo médio, volume de interações)
3. um modelo supervisionado simples classifica interações
4. o dashboard exibe tudo visualmente em tempo real

Tudo isso compõe o pipeline completo de dados da Sprint 2.

4. Módulos Técnicos (Descrição Completa)

4.1 Módulo 1 — Simulador de Sensores

Arquivo: sprint2/sensor_simulation/simulate_sensor.py

Função do módulo

Simula o comportamento de um totem real, gerando dados contínuos:

- presença diante do totem
- toque na interface
- comandos de voz
- tempo de permanência
- frequência definida (5 segundos)

Por que isso é importante?

Permite testar o fluxo completo sem precisar montar o hardware real nesta etapa inicial.

```
(.venv) PS C:\Users\mathe\Desktop\matheus_meissner\dev\inclusive_culture_totem\sprint2\sensor_simulation> python simulate_sensor.py
Iniciando simulador de sensor do Totem Flexmedia...
Banco de dados: ../database/totem.db
Interação registrada: {'timestamp': '2025-11-23T11:17:27', 'presence': 0, 'touch': 0, 'voice_detected': 0, 'duration': 0}
Interação registrada: {'timestamp': '2025-11-23T11:17:32', 'presence': 1, 'touch': 0, 'voice_detected': 1, 'duration': 10}
Interação registrada: {'timestamp': '2025-11-23T11:17:37', 'presence': 0, 'touch': 0, 'voice_detected': 0, 'duration': 0}
Interação registrada: {'timestamp': '2025-11-23T11:17:42', 'presence': 1, 'touch': 0, 'voice_detected': 1, 'duration': 9}
Interação registrada: {'timestamp': '2025-11-23T11:17:47', 'presence': 1, 'touch': 1, 'voice_detected': 0, 'duration': 11}
Interação registrada: {'timestamp': '2025-11-23T11:17:52', 'presence': 1, 'touch': 1, 'voice_detected': 1, 'duration': 12}
Interação registrada: {'timestamp': '2025-11-23T11:17:57', 'presence': 0, 'touch': 0, 'voice_detected': 0, 'duration': 0}
Interação registrada: {'timestamp': '2025-11-23T11:18:02', 'presence': 0, 'touch': 0, 'voice_detected': 0, 'duration': 0}
Interação registrada: {'timestamp': '2025-11-23T11:18:07', 'presence': 1, 'touch': 1, 'voice_detected': 1, 'duration': 8}
Interação registrada: {'timestamp': '2025-11-23T11:18:12', 'presence': 0, 'touch': 0, 'voice_detected': 0, 'duration': 0}
Interação registrada: {'timestamp': '2025-11-23T11:18:17', 'presence': 0, 'touch': 0, 'voice_detected': 0, 'duration': 0}
Interação registrada: {'timestamp': '2025-11-23T11:18:22', 'presence': 0, 'touch': 0, 'voice_detected': 0, 'duration': 0}
Interação registrada: {'timestamp': '2025-11-23T11:18:27', 'presence': 0, 'touch': 0, 'voice_detected': 0, 'duration': 0}
```

4.2 Módulo 2 — Armazenamento SQL (SQLite)

Pasta: sprint2/database/

Arquivo: totem.db (gerado automaticamente)

A Sprint 2 pede um banco SQL simples.

A tabela utilizada:

```
CREATE TABLE interactions (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    timestamp TEXT NOT NULL,  
    presence INTEGER NOT NULL,  
    touch INTEGER NOT NULL,  
    voice_detected INTEGER NOT NULL,  
    duration INTEGER NOT NULL  
);
```

Por que SQLite?

- é leve
- não depende de servidor
- funciona localmente para demonstração
- atende 100% o requisito da sprint

4.3 Módulo 3 — Análise de Dados (Notebook)

Arquivo: analysis.ipynb

Funções principais:

- leitura do banco de dados
- preparação das variáveis
- análise temporal (dia / hora / timeline)
- análise descritiva das interações
- criação de categorias baseadas na duração
- geração de gráficos
- preparação dos dados para ML

Prints demonstrativos:

Célula 1 – Imports e Conexão

[2] ✓ 10.1s

...

	id	timestamp	presence	touch	voice_detected	duration
0	1	2025-11-22T14:33:34	0	0	0	0
1	2	2025-11-22T14:33:39	0	0	0	0
2	3	2025-11-22T14:33:44	1	0	0	4
3	4	2025-11-22T14:33:49	1	1	1	18
4	5	2025-11-22T14:33:54	0	0	0	0

Célula 2 – Conversão de tipos e limpeza básica

✓ 0.0s Python

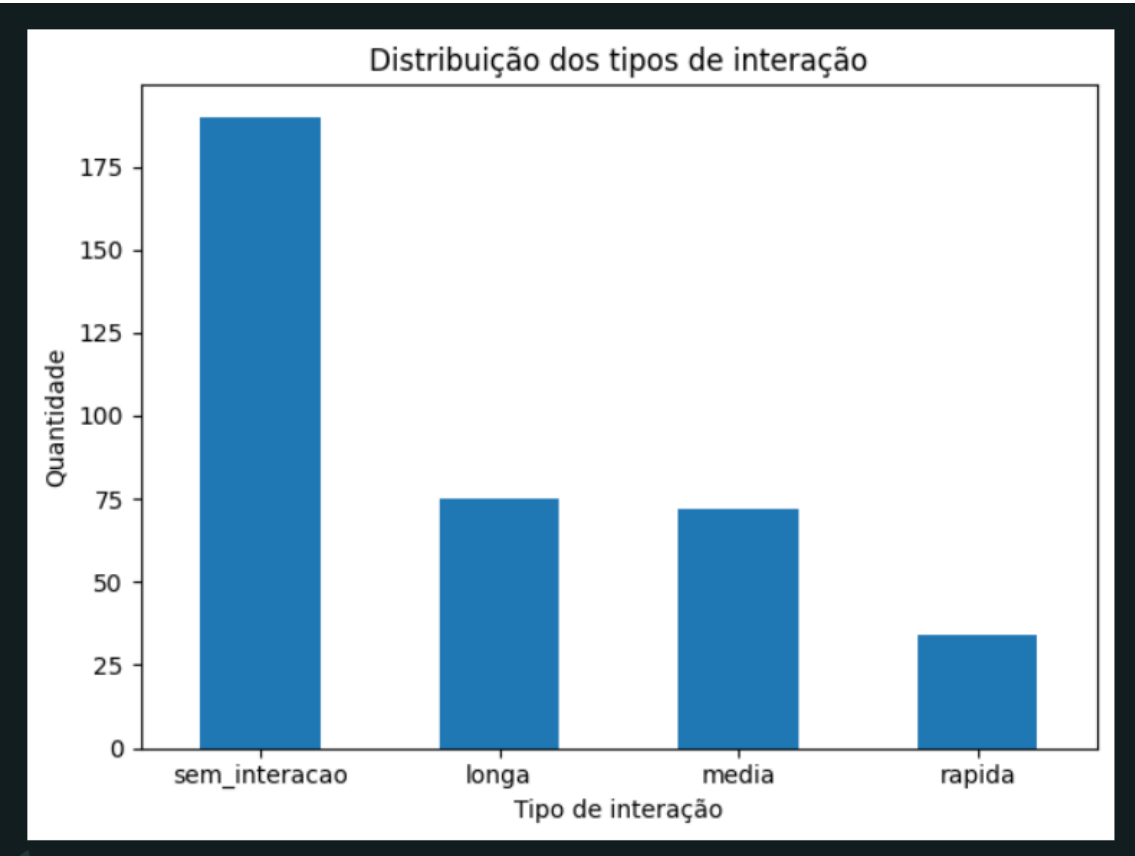
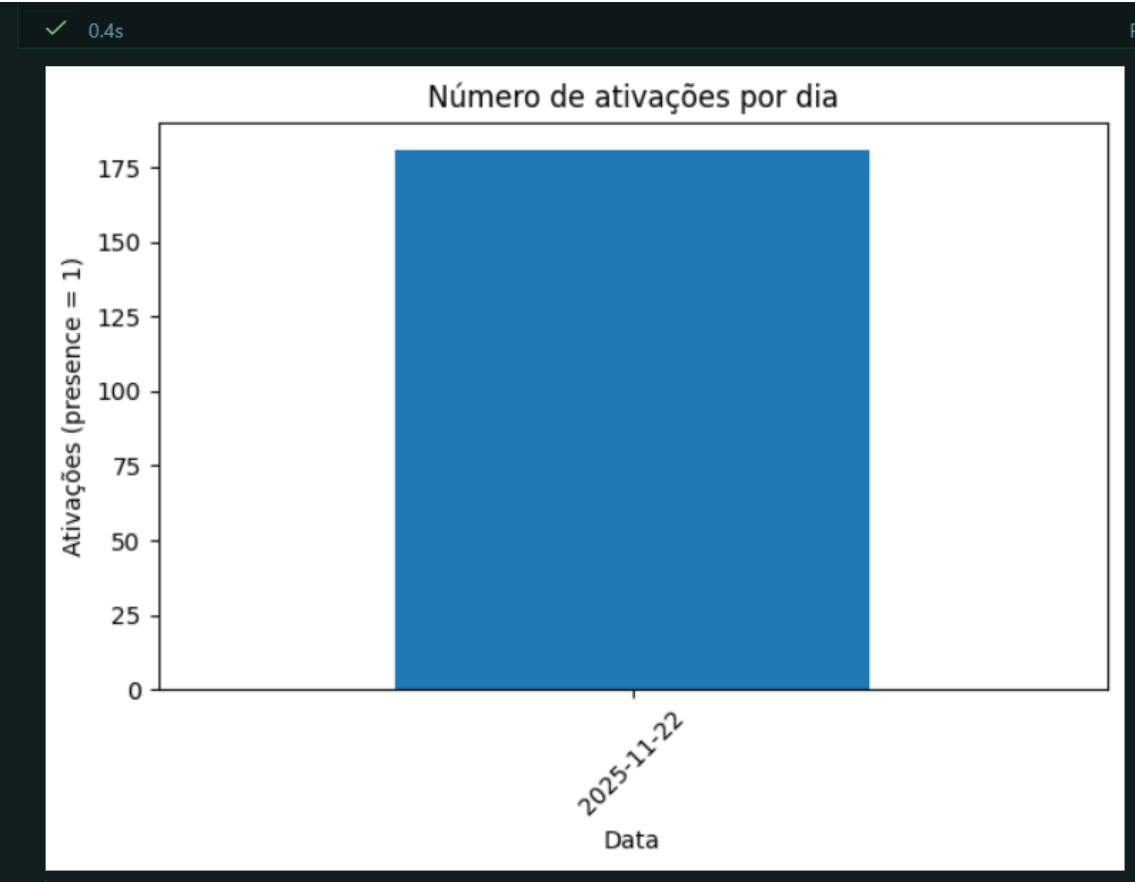
	id	timestamp	presence	touch	voice_detected	duration
count	371.000000	371	371.000000	371.000000	371.000000	371.000000
mean	186.000000	2025-11-22 14:53:44.371968	0.487871	0.207547	0.218329	5.145553
min	1.000000	2025-11-22 14:33:34	0.000000	0.000000	0.000000	0.000000
25%	93.500000	2025-11-22 14:41:17.500000	0.000000	0.000000	0.000000	0.000000
50%	186.000000	2025-11-22 14:49:01	0.000000	0.000000	0.000000	0.000000
75%	278.500000	2025-11-22 15:07:29.500000	1.000000	0.000000	0.000000	10.000000
max	371.000000	2025-11-22 15:15:13	1.000000	1.000000	1.000000	20.000000
std	107.242715	NaN	0.500528	0.406098	0.413670	6.572700

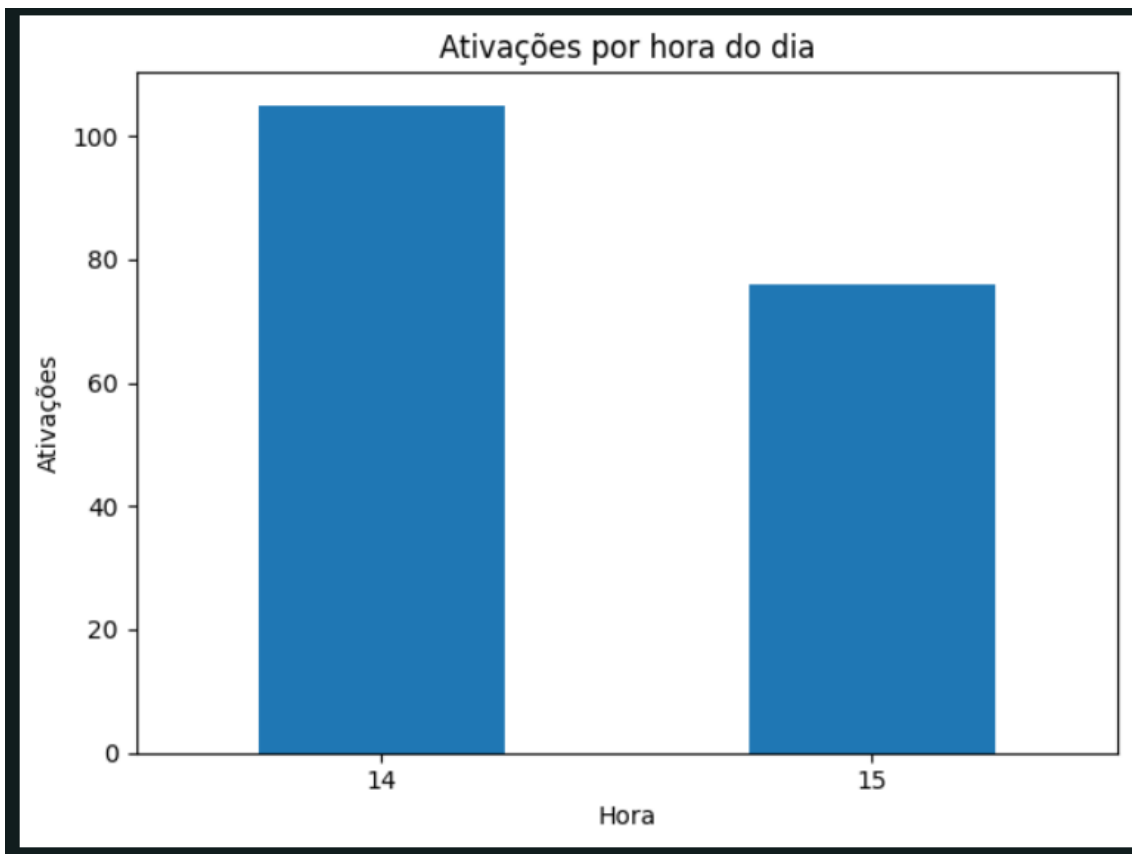
Célula 3 – Feature engineering e métricas descritivas

✓ 0.0s Python

	id	timestamp	presence	touch	voice_detected	duration	houve_interacao	tipo_interacao
0	1	2025-11-22 14:33:34	0	0	0	0	nao	sem_interacao
1	2	2025-11-22 14:33:39	0	0	0	0	nao	sem_interacao
2	3	2025-11-22 14:33:44	1	0	0	4	sim	rapida
3	4	2025-11-22 14:33:49	1	1	1	18	sim	longa
4	5	2025-11-22 14:33:54	0	0	0	0	nao	sem_interacao

Célula 4 – Análises por horário e gráficos básicos





Célula 5 - Modelo de Machine Learning supervisionado

```
✓ 0.0s

Relatório de classificação:

              precision    recall  f1-score   support

   nao_rapida      1.00      1.00      1.00        43
     rapida      1.00      1.00      1.00        12

   accuracy              1.00          55
  macro avg      1.00      1.00      1.00          55
weighted avg      1.00      1.00      1.00          55

Matriz de confusão:

[[43  0]
 [ 0 12]]
```

Célula 6 – Salvando um resumo para o dashboard

 0.0s

	data	ativacoes	media_duracao
0	2025-11-22	181	5.145553

4.4 Módulo 4 — Machine Learning Supervisionado

Modelo: **RandomForestClassifier**

Objetivo: **classificar interações como rápidas × não rápidas**

Features utilizadas:

- presence
- touch
- voice_detected
- duration

Saídas:

- acurácia
- matriz de confusão
- relatório de classificação

classification_report e matriz de confusão

```
✓ 0.0s

Relatório de classificação:

              precision    recall  f1-score   support

   nao_rapida      1.00      1.00      1.00        43
     rapida      1.00      1.00      1.00        12

   accuracy              1.00              55
  macro avg      1.00      1.00      1.00              55
weighted avg      1.00      1.00      1.00              55

Matriz de confusão:

[[43  0]
 [ 0 12]]
```

Isso atende ao requisito:

“Aplicar um exemplo simples de Machine Learning supervisionado usando o dataset da equipe.”

4.5 Módulo 5 — Dashboard em Streamlit

Arquivo: dashboard/app.py

KPIs exibidos:

- registros totais
- ativações
- duração média
- total de interações com permanência

Gráficos:

- ativações ao longo do tempo
- tipos de interação
- ativações por dia
- ativações por hora

Recursos:

- atualização em tempo real
- leitura direta do SQLite
- tabela bruta dos últimos registros

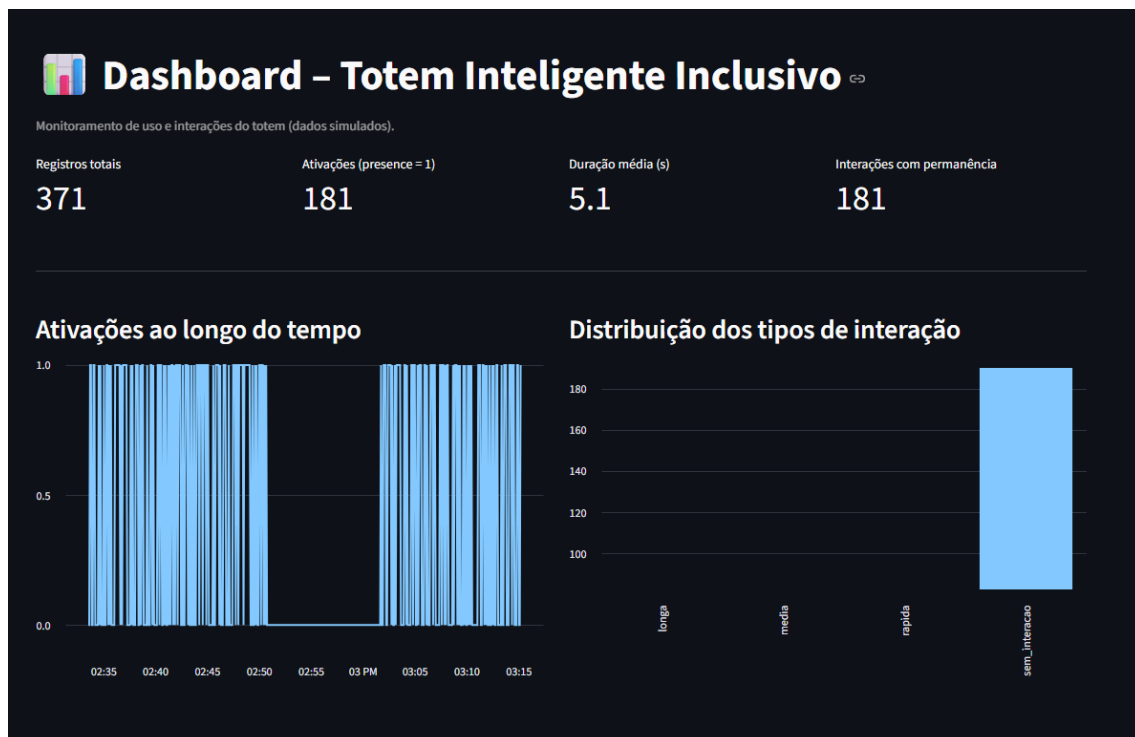




Tabela de dados bruta											
	id	timestamp	presence	touch	voice_detected	duration	houve_interacao	tipo_interacao	data	hora	
321	322	2025-11-22 15:11:08	1	0	0	7	sim	media	2025-11-22	15	
322	323	2025-11-22 15:11:13	1	0	1	4	sim	rapida	2025-11-22	15	
323	324	2025-11-22 15:11:18	1	1	1	14	sim	longa	2025-11-22	15	
324	325	2025-11-22 15:11:23	0	0	0	0	nao	sem_interacao	2025-11-22	15	
325	326	2025-11-22 15:11:28	0	0	0	0	nao	sem_interacao	2025-11-22	15	
326	327	2025-11-22 15:11:33	1	0	0	14	sim	longa	2025-11-22	15	
327	328	2025-11-22 15:11:38	1	0	0	14	sim	longa	2025-11-22	15	
328	329	2025-11-22 15:11:43	0	0	0	0	nao	sem_interacao	2025-11-22	15	
329	330	2025-11-22 15:11:48	1	0	0	7	sim	media	2025-11-22	15	
330	331	2025-11-22 15:11:53	0	0	0	0	nao	sem_interacao	2025-11-22	15	

5. Conclusão Técnica da Sprint 2

Nesta sprint, estabeleci o pipeline completo que conecta sensores (simulados), armazenamento SQL e análise inteligente.

Com isso, o Totem Inteligente Inclusivo já é capaz de:

- ✓ registrar uso
- ✓ processar dados
- ✓ gerar métricas
- ✓ treinar um modelo simples de ML
- ✓ exibir indicadores e gráficos em um dashboard funcional

Esta estrutura é a base essencial para o avanço da Sprint 3, que introduzirá camadas mais profundas de inteligência e recomendações.